

02 | 负载均衡：调用结果、缓存机制是怎么影响负载均衡的？

2023-6-19 邓明



你好，我是大明。今天我们来聊一聊微服务架构下的负载均衡。

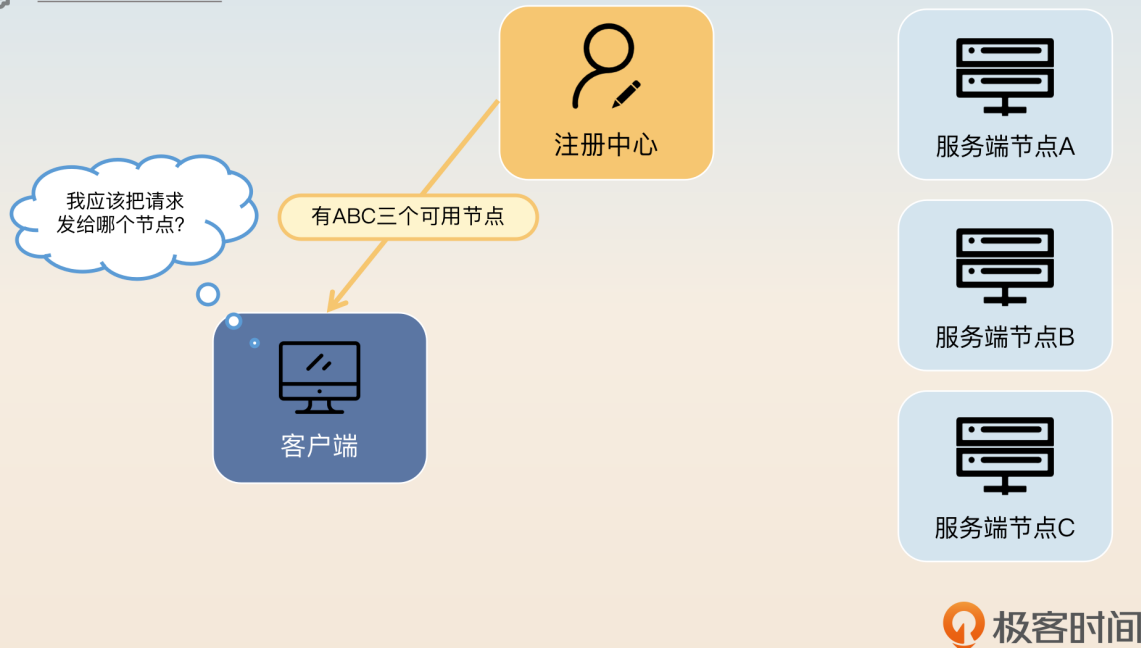
负载均衡在微服务架构里也处于一个核心位置。一般我们在准备调用任何服务的时候，第一个要解决的问题就是负载均衡该怎么做。负载均衡在微服务架构的面试中，也属于必面题目。

可惜的是，即便我们都知道负载均衡在面试中是必考点，但是在每一次面试的时候都还是难以刷出亮点。大多数的回答都仅仅是简单罗列一下负载均衡的算法，稍微有些亮点的则是讨论一下不同算法的优缺点。但是这并不能让你在面试官心里留下深刻印象。

所以今天我就来给你介绍一下负载均衡算法里面一些可以用于面试的微妙细节，同时给出一个本地缓存和负载均衡结合的案例，让你在面试的时候刷出亮点。下面我先来给你介绍微服务架构里面常见的负载均衡算法，让你先有一个最基本的理解。

前置知识

负载均衡，本质上就是回答一个问题：**我该把请求发给哪个服务端？**理论上来说，你会希望把请求发给某个能够最快返回响应的客户端。



这里你可能会觉得有些困惑，因为我们之前都听过轮询和加权轮询、随机和加权随机，哈希和一致性哈希这些负载均衡算法，但看上去它们并没有试图去判断哪个节点才是最合适的节点。

确实，这一类算法也叫做**静态负载均衡算法**。它们依靠的是统计学上的“最合适”。也就是说，如果请求都差不多，请求数量也足够多，那么它们能够挑选出比较合适的节点。

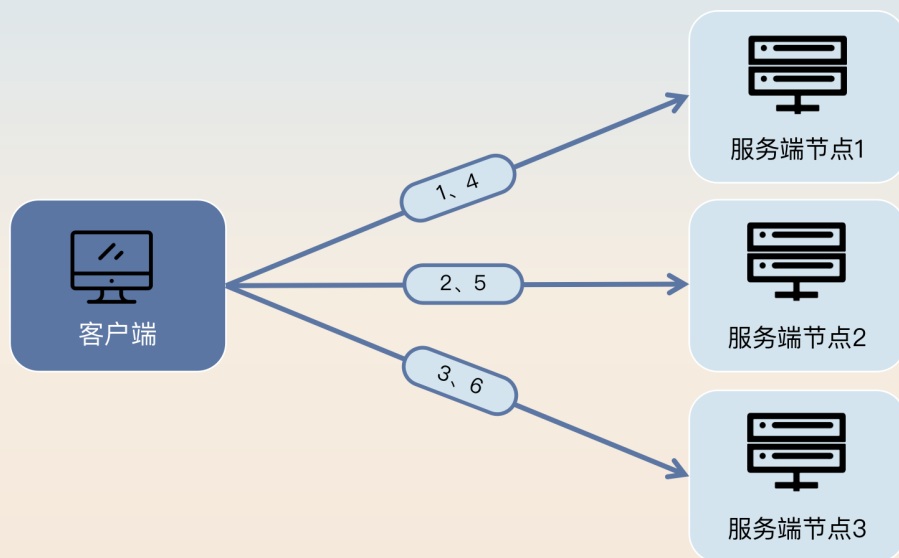
还有一类算法，是**动态负载均衡算法**，或者说是实时检测负载均衡算法。这一类算法依赖于实时判断所有候选节点的状态，并且从里面挑选出最合适的节点。这一类算法包含最少连接数、最少活跃请求数、最快响应时间等算法。

接下来我带你一个个看。

轮询与加权轮询

轮询本身是一个非常简单的算法，用一句俗话讲，就是“排排坐，分果果”。也就是说，所有的候选节点轮流作为负载均衡的目标节点。

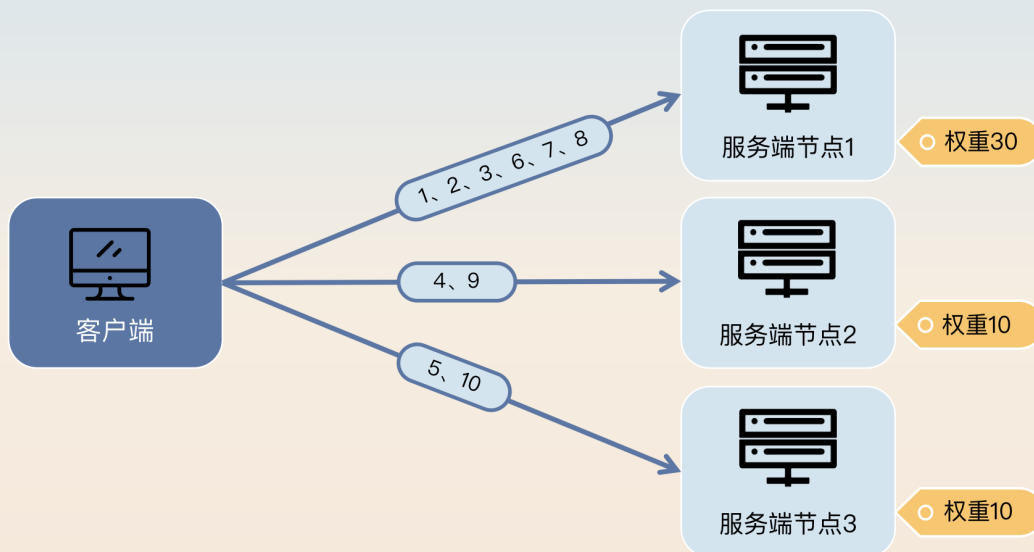
💡 轮询



极客时间

你可能想到，每个节点的实际处理能力可能并不一样。于是就有了一个加权的版本，就是所谓的加权轮询。这个算法就不再是节点轮流，而是**根据权重来轮流**。比如说，如果一个节点的权重是另外一个节点的两倍，那么最终这个节点被选中的次数也会是另外一个节点的两倍。

💡 加权轮询



极客时间

图中节点1的权重是其他两个节点的三倍，所以相应地被选中的机会也是三倍。

在加权算法里面，有一个改进叫做平滑的加权轮询算法。你在图里也可以看出来，因为服务端节点1的权重是别的节点的三倍，所以如果你不做任何措施，那么会连续三次将请求发送到同一个节点。

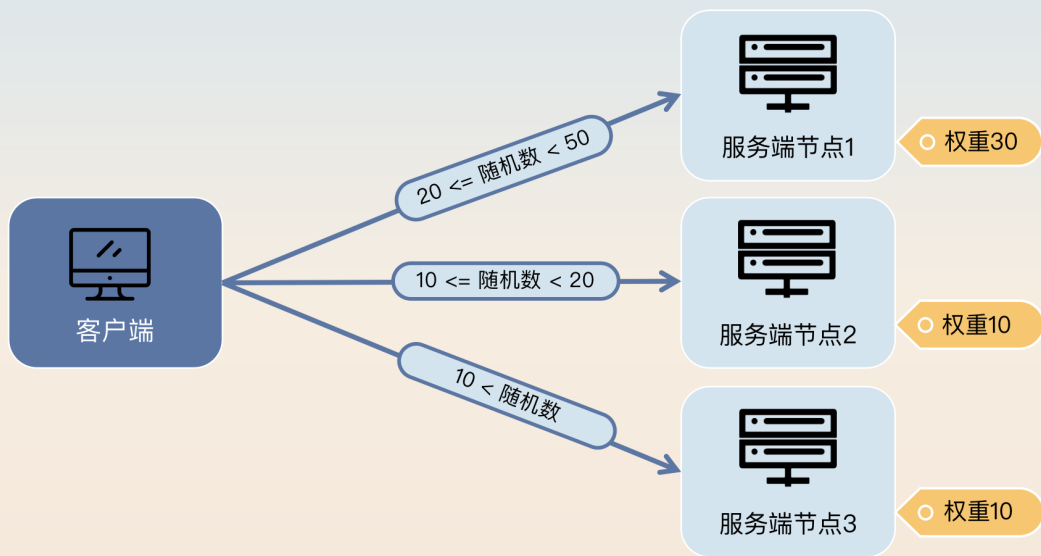
而这个平滑的加权轮询算法就是为了解决这个问题。每个节点会有两个权重，初始权重（weight）和当前权重（currentWeight）。算法的过程稍微有点复杂，每一次挑选节点都执行这些步骤。

1. 对每一个节点，执行 $\text{currentWeight} = \text{currentWeight} + \text{weight}$ 。
2. 挑选最大 currentWeight 的节点作为目标节点。
3. 将目标节点的 currentWeight 修改为 $\text{currentWeight} = \text{currentWeight} - \text{sum}(\text{weight})$ 。

那么简单理解就是，对于一个节点来说，每次被挑选之后，它的 currentWeight 就会下降，那么下一次就不会选中它。

随机与加权随机

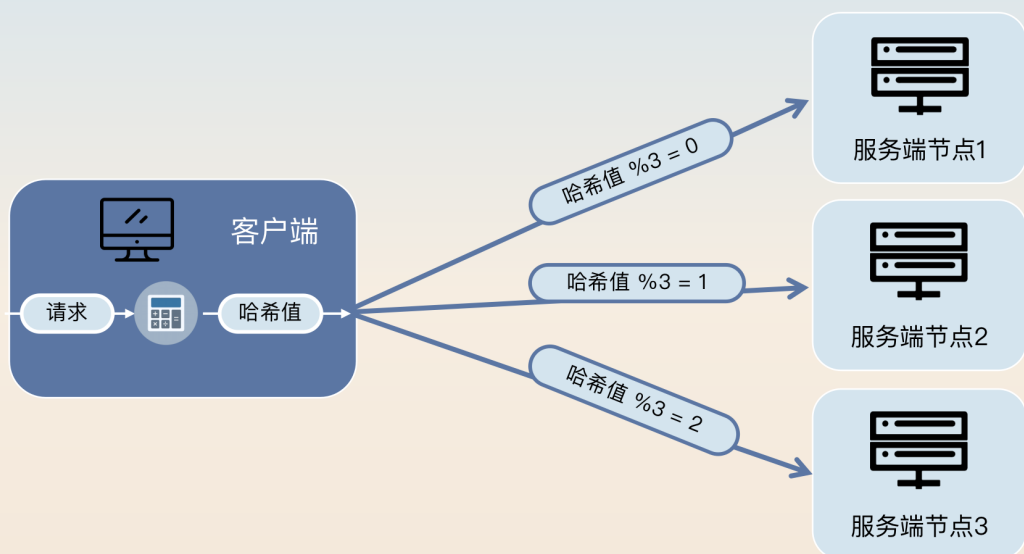
随机可以看作是随便挑选一个作为目标节点，加权随机则是利用不同的权重来设置选中的概率。权重越大，那么被选中的机会也就越大。



你可以看到，轮询算法相比之下可控性更强。一般来说，在实践中轮询和随机、加权轮询和加权随机是可以互相替代的。

哈希与一致性哈希

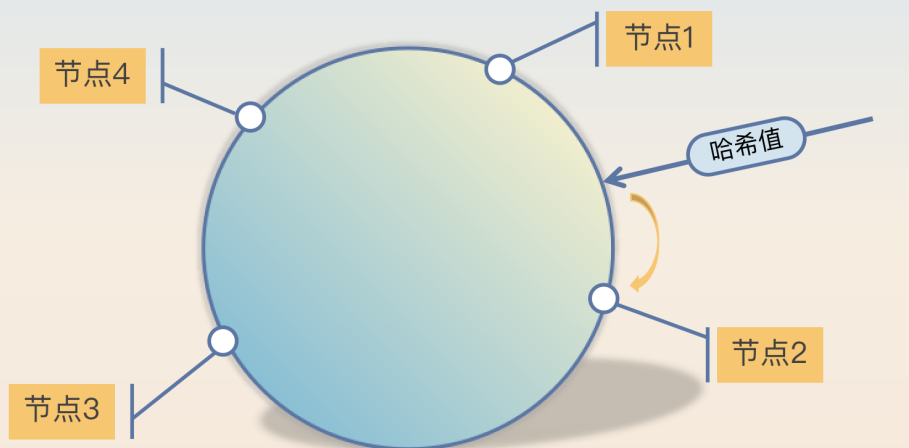
哈希算法比较简单，一般就是选取请求里面某几个参数来计算一个哈希值，然后除以节点数量取余。这个过程几乎和随机一样，区别就在于随机算法里面用的是随机数，这里用的是根据参数计算出来的哈希值。



哈希算法的选取会严重影响负载均衡的效果。假如说你计算哈希值的算法不太好，就容易导致某几个节点上负载特别高，而其他节点的负载就比较低。所以要尽可能保证哈希值计算出来的结果是均匀的。

相比之下一**一致性哈希负载均衡**才算是真正的面试热点。

一致性哈希负载均衡引入了一个哈希环的概念，服务端节点会落在环的某些位置上。客户端根据请求参数，计算一个哈希值。这个哈希值会落在哈希环的某个位置。从这个位置出发，顺时针查找，遇到的第一个服务端节点就是目标节点。



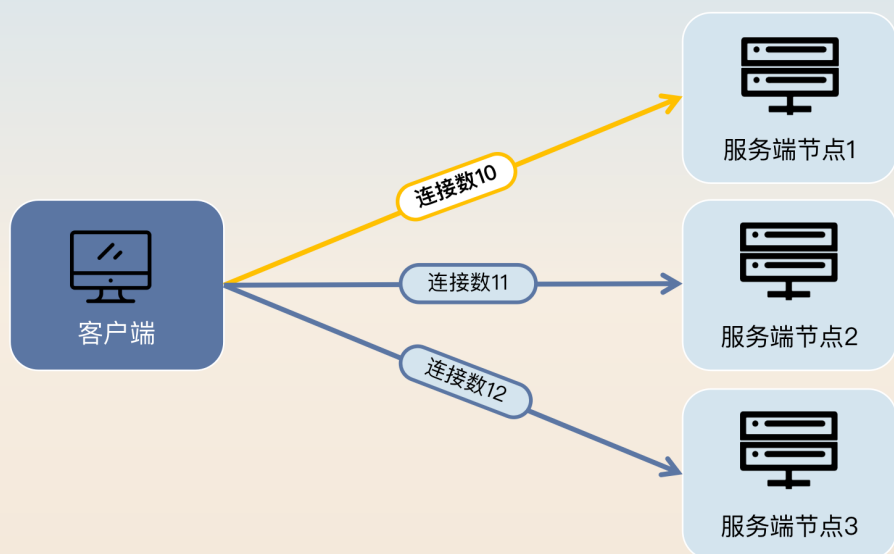
注意，在一致性哈希负载均衡算法里面，并不要求服务端节点是均匀分散在哈希环上的。实际上，我们是希望所有的节点负载是均衡的，但是不同节点之间的间隔可以是不均匀的。

这里我可以用一个比喻来简单描述这个算法。一致性哈希负载均衡算法就像是钟表，它的过程就有点儿像你的朋友约你吃火锅，说下一个整点到重庆火锅店集合。那么你看一下现在的时间，是下午三点四十五分，那么自然下一个整点就是下午四点了。

在面试的时候，如果你实在记不住一致性哈希负载均衡算法，那么可以用这个比喻来向面试官解释一下。

最少连接数

最少连接数基于一个基本假设：如果一个服务端节点上的连接数越多，那么这个节点的负载就越高。因此在做负载均衡的时候就是看一下客户端和各个节点的连接数量，从中挑选出连接数数量最少的节点。

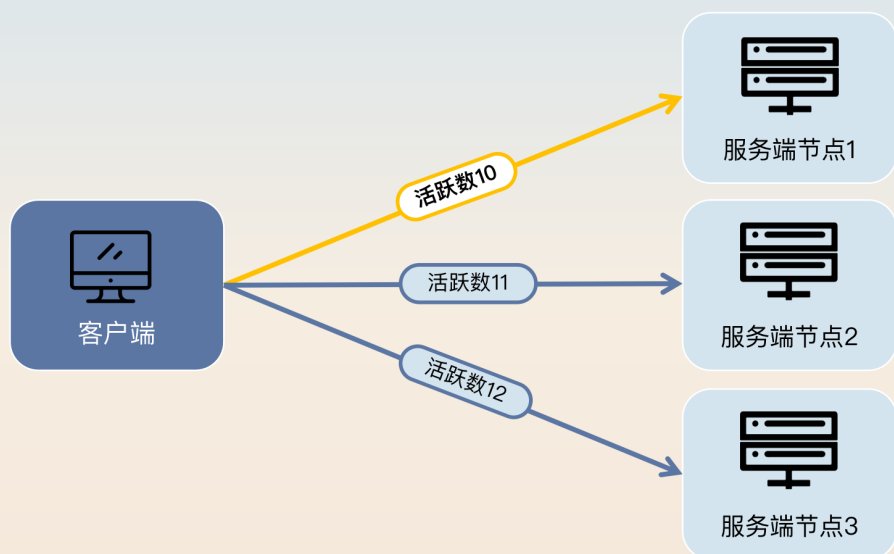


最少连接数算法的缺陷在于，连接数并不能代表节点的实际负载，尤其是在连接多路复用的情况下。比如这张示意图里，理论上来说新来的请求就会落到服务端节点 1 上，而后连接数变成 11。实际上在连接复用的情况下，客户端可能连续发 10 个请求到服务端节点 1 上，才会创建一个新连接。

那么很显然，在这种情况下，服务端节点 1 的负载会比其他两个节点高一截。

最少活跃数

最少活跃数算法则是用当前活跃请求数来代表服务端节点的负载。所谓的活跃请求，就是已经接收但是还没有返回的请求。客户端会维持一个自己发过去但是还没返回的请求数量，然后每次挑选活跃请求最少的那个服务端节点。

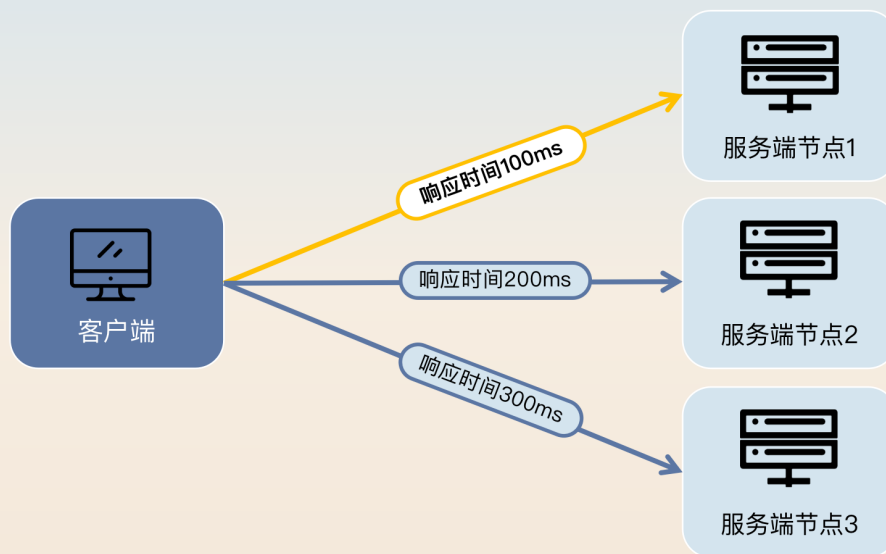


类似地，活跃请求数量也不能真正代表服务端节点的负载。比如说图中，服务端节点1虽然只有10个请求，但是万一这10个请求都是大请求，例如大商家、大买家或者千万粉丝UP主的请求，那么服务端节点1的负载也会显著高于其他两个节点。

最快响应时间

相比前两个算法，最快响应时间算法则要好很多，它用的是响应时间来代表服务端节点的负载。响应时间和前面的两个指标比起来，是一种综合性的指标，所以用响应时间来代表服务端节点负载要更加准确。

最快响应时间算法就是客户端维持每个节点的响应时间，而后每次挑选响应时间最短的。



这里的响应时间，可以是平均响应时间，也可以是 99 线之类的，选择什么其实效果并不会相差很多。

但是在实现上，要注意响应时间的时效性。一般来说统计响应时间时应该只用近期请求的响应时间，并且越近的响应时间，权重应该越高。换句话说，就是采集的响应时间效用应该随着时间衰减。

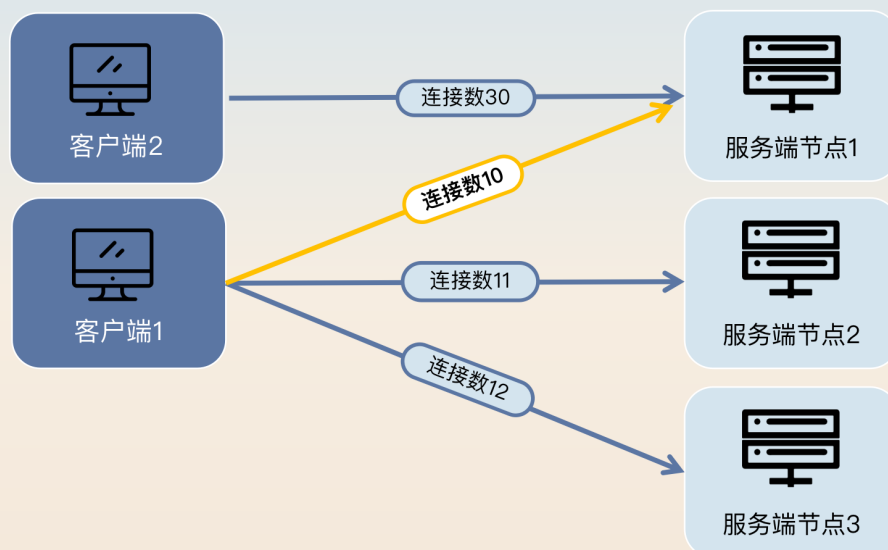
小结一下

最少连接数、最少活跃请求数和最快响应时间，都可以看作是选择了单一的指标来代表一个节点的负载。

那么你在实际工作中也可以利用这个思路来设计自己的负载均衡算法。比如说在 CPU 密集型的应用里面，你可以设计一个负载均衡算法，每次筛选 CPU 负载最低的节点。难点则是你需要考虑**怎么采集到所有服务端节点的 CPU 负载数据**。

这三个算法还有一个问题，就是它们都是客户端来采集数据的。**那么不同的客户端就可能采集到不同的数据**。如图所示，因为客户端 1 本身并不知道客户端 2 上还有 30 个连接，因此它选择了服务端节点 1。而实际上它应该选择服务端节点 2。

💡 客户端1选择了错误节点



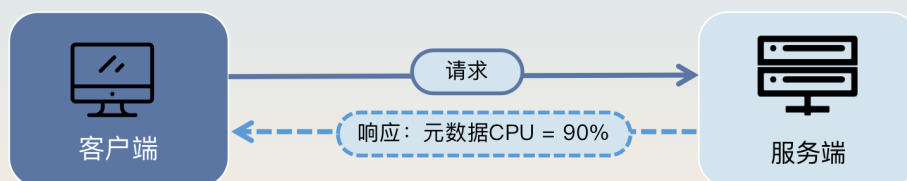
极客时间

那怎么解决这两个问题呢？

答案是让服务端上报指标，而不是客户端采集。总体上有两种思路。

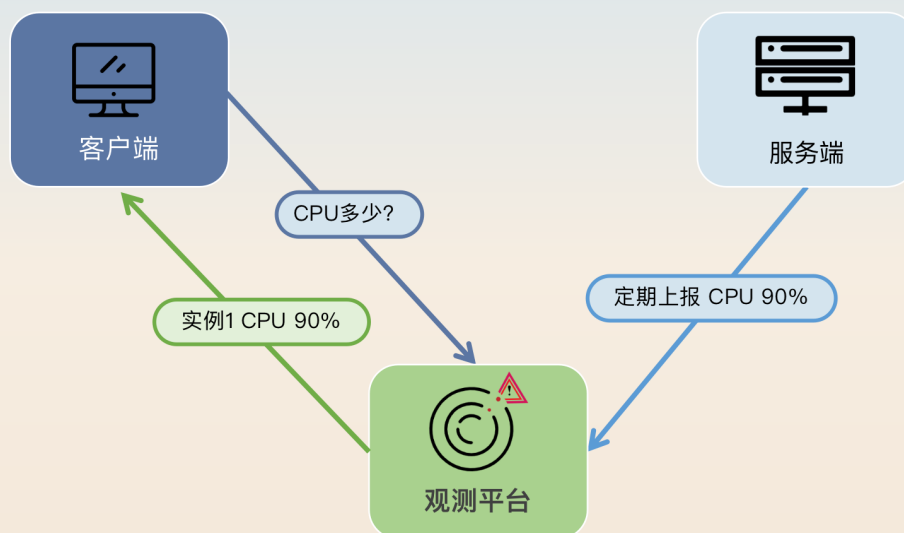
第一种思路是服务端在返回响应的时候顺便把服务端上的一些信息一并返回。这种思路需要微服务框架支持从服务端往客户端回传链路元数据。

💡 第一种思路：响应中携带采集结果



极客时间

第二种思路是从观测平台上查询。例如通过查询Prometheus来获得各种指标数据。



不过目前业界很少用这种复杂的负载均衡算法，也因此几乎所有的微服务框架都没有服务端上报指标到客户端的机制。

面试准备

首先我前面提到的这些算法，你都要记下来。尤其要对这几个算法格外上心一些。

轮询和加权轮询：对应的平滑加权轮询算是一个小亮点。

一致性哈希负载均衡：这个可以结合 Redis 之类的使用了一致性哈希算法的中间件一起理解。

最快响应时间算法：这个算法体现了采集指标随着时间准确性衰减的特性，后面在服务治理的部分你会再次接触到类似的东西。

那么我们对这些算法的简单分析你也要记住，尤其是在小结里提到的**采集指标**的问题。然后你在准备项目经验的时候要搞清楚公司以下几种情况。

如果公司有 Nginx 之类的网关，或者微服务网关，那么用的是什么负载均衡算法？

如果公司用客户端负载均衡的话，用的是什么负载均衡算法？

有没有出过和负载均衡相关事故，如果有，那么是什么原因导致的，怎么解决的这个事故，它体现了负载均衡算法的什么缺陷？

另外，你还可以尝试根据业务设计一个独一无二的负载均衡算法。即便你用的是最简单的轮询之类的算法，也不用担心。因为目前大规模应用的就是这种简单的算法，那些花里胡哨的算法在面试和汇报晋升的时候很有用，但是实际上落地的并不多。

如果你现在有足够的时间，那么前面这些算法你都可以试着实现一下。我尤其建议你在 gRPC 里接入一下自己写的算法，做个小实验。一方面是加深理解，另一方面是防止面试官要求你现场写算法实现。

小结中讨论的内容可以作为一个亮点，但是稍微有点理论化，所以你还需要掌握一些实践方面的答题亮点。

怎么根据调用结果来调整权重，从而影响负载均衡的效果？

怎么利用一致性哈希负载均衡算法，来提高本地缓存命中率，缓解数据不一致性问题？

这两个问题你可以从后面找找答案。

亮点方案

正常来说，面试官会先问你“是否了解负载均衡”“知道哪些负载均衡算法”之类的问题，那么你就可以列举前置知识里面提到的算法，然后要结合自身公司的实际情况，说明自己用的是什么负载均衡算法。

如果你准备了一个负载均衡引发的线上事故案例，那么一定要记得展开聊一聊。这里我用轮询作为例子，你可以参考这个例子来准备。

首先在回答里面你要先描述各种基本算法以及简要分析，然后再加上一句总结引导。

一般来说，加权类的算法都要考虑权重的设置和调整。

紧接着你开始说你们公司的负载均衡算法，关键词是**大请求**，你可以这么说。

我们公司用的是轮询来作为负载均衡。不过因为轮询没有实际查询服务端节点的负载，所以难免会出现偶发性的负载不均衡的问题。

比如说我们之前发现线上的响应时间总体来说是非常均匀的，但是每隔一段时间就会出现响应时间特别慢的情况。而且时间间隔是不固定的，慢的程度也不一样，所以就很奇怪。后来我们经过排查之后，发现是因为当一个大请求落到一个节点的时候，它会占据大量的内存和 CPU。如果这时候再有请求打到同一个节点上，这部分请求的响应时间就会非常慢。

在这个回答里我用的这个例子说明了所有负载均衡算法都有的缺点，即**没有考虑请求本身**。一个大商家拉当日成交订单数据，和一个长尾商家拉当日成交订单数据，能是一回事吗？显然不是。

在这个例子里面，我并没有说怎么解决问题，其实这也是在引导面试官进一步问。如果他问如何解决，那么你可以从**业务拆分**或者**隔离**的角度回答。

（业务拆分）这个大请求其实是一个大的批量请求。后来我们限制一批最多只能取100个就解决了这个问题。

（隔离角度）我们稍微魔改了一下负载均衡算法，不再是单纯的轮询了。我们每天计算一批大客户，这部分大客户的请求会在负载均衡里面被打到专门的几个节点上。虽然大客户的请求依旧很慢，但是至少别的客户不会再受到他们的影响了。

隔离角度的回答相比之下会更加高级一点，因为你可以借此机会将这个回答引到服务治理中的隔离措施这个话题上。同时，这个角度还体现了一个魔改负载均衡的创新点。但如果你不太熟悉服务治理类的话题，那么用业务拆分的角度来回答会更加合适。

这时候你可以补上一句总结，升华一下回答。

负载均衡算法有些时候用得好，是能够解决一些技术问题的，比如说缓存。

这里你应该能够看出来，不论是刚刚说的“加权类的算法都要考虑权重设置和调整”还是这个“能够解决一些技术问题的”，都是在钓鱼。这两个点也就是你能彻底拉开和其他候选者

差距的亮点。

调用结果对负载均衡的影响

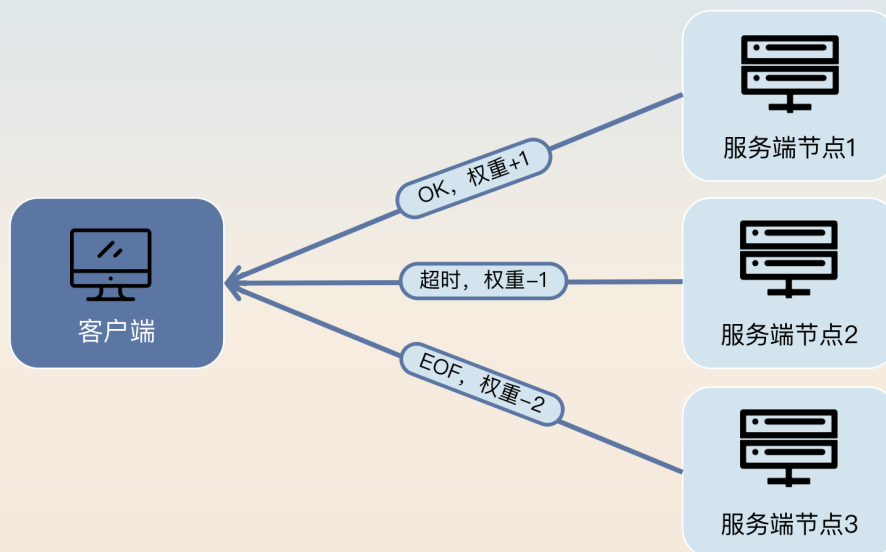
前面我和你提到过在负载均衡里面有对应的加权版本，比如说轮询有对应的加权轮询版本，随机也有对应的加权随机版本。

而实际上在工作中我们可以考虑根据调用结果来动态调整这个权重。所以如果面试官问你怎么设置权重或者怎么调整权重，抓住关键词**成加败减**就可以了。

权重代表节点的处理能力，当然在一些场景下它也代表节点的可用性或者重要性。所以权重根据节点的实际情况来设置值就可以。权重的要点在于体现不同节点的差异性，它的绝对值并不重要。

一般来说为了进一步提高可用性，加权类的负载均衡算法都会考虑根据调用结果来动态调整权重。如果调用成功了，那么就增加权重；如果调用失败了，那么就减少权重。

这里调用成功与否是一种非业务相关的概念，也就是说即便拿到了一个失败的响应，但是本身也算是调用成功了。调用失败了大多数时候是指网络错误、超时等。而在实际落地的时候，也可以考虑如果是网络引起的失败，那么权重下调就多一点，因为这一类的错误意味着问题更加严重。如果是超时这种，那么权重就下调少一点，因为这种错误是比较容易恢复过来的。



我这里稍微给你解释一下为什么说绝对值不重要。比如说一个方案是 A 的权重是 100，B 的权重是 200，与另一个方案 A 的权重是 1000，而 B 的权重是 2000，负载均衡的效果是一样的。但是在调整权重的时候要按比例来调。比如说前一个方案调整权重可能每次调 10，而后一个方案就要每次调 100。

回答到这里，还有一个很多开发者都意识不到以至于经常有人踩坑的点：**权重的调整要设置好上限和下限**。那么你可以揭开这个业界经常忽略的问题，关键词是**上下限**。

调整权重的算法都要考虑安全问题，即权重的调整应该有上限和下限。比如说一般下限不能为 0，因为一个节点的权重为 0 的话，它可能永远也不会被选中，又或者和 0 的数学运算会出现问题导致负载均衡失败。上限一般不超过初始权重的几倍，比如说两倍或者三倍，防止该节点一直被连续选中。

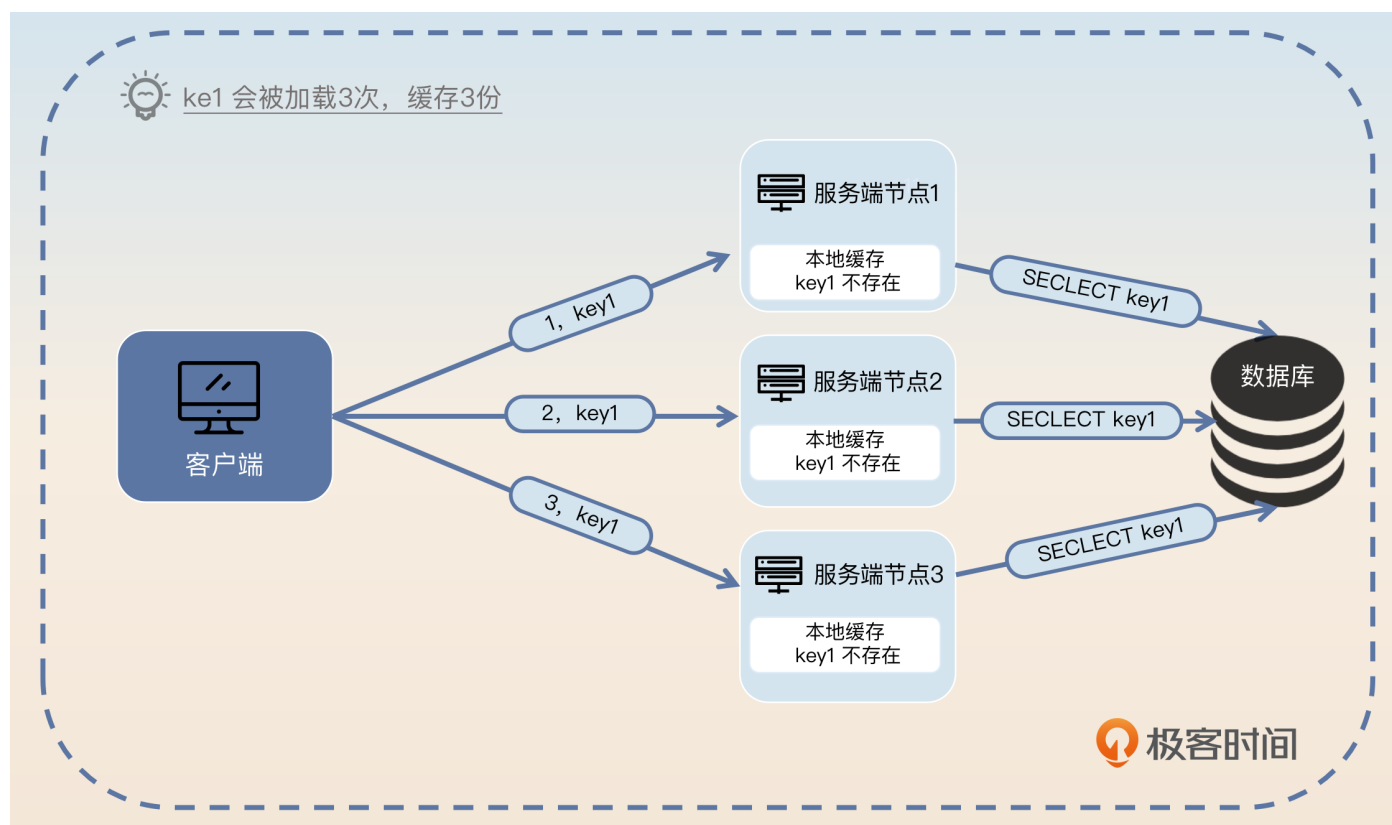
当然，如果在实现的时候使用了 uint 或者 int8 之类的数字，还要进一步考虑溢出的问题。之前挺多公司因为没有控制上下限而引起了线上故障。

这里你如果对服务注册与发现烂熟于心，那么就可以尝试将话题引导到服务注册与发现中，关键词是**可用性**。

这种根据调用结果来调整权重的方式，有点类似于在服务中将暂时调用不通的节点挪出可用节点列表，本质上都是为了进一步提高系统的可用性。

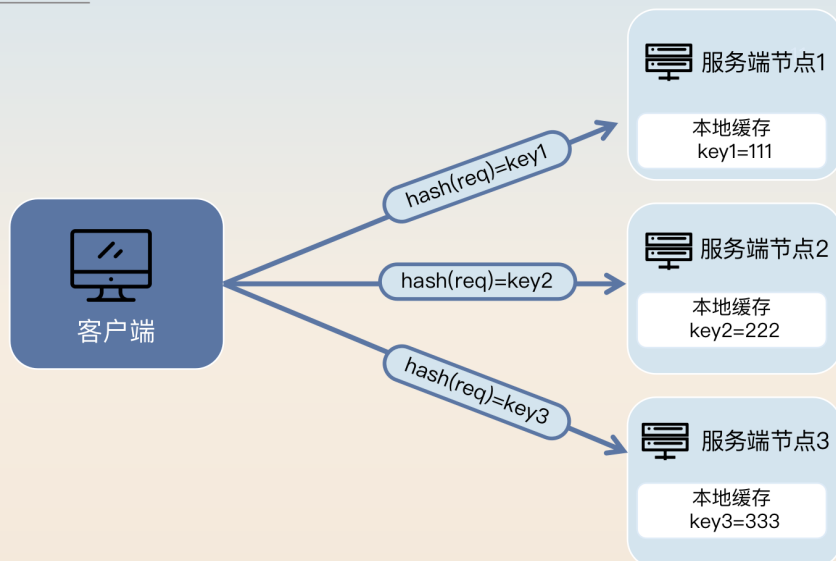
哈希一致性结合本地缓存

这算是一个微创新的方案。正常情况下，如果你使用本地缓存，那么同一个 key 对应的请求，可能会被打到不同的节点上。这就会造成两个问题，一个是严重的缓存未命中，一个是不同节点都要缓存同样的数据，导致内存浪费和极其严重的数据一致性问题。



所以在这种情况下，一个很自然的想法就是能不能把类似的请求都让同一个节点来处理。比如说对某个用户数据的请求都打到同一个节点上。

显然适合的负载均衡算法就两个：哈希或者一致性哈希。



如果我们考虑节点可能上线、下线的情况，那么一致性哈希负载均衡就是最优选择。所以你可以先简单介绍一下方案。

在性能非常苛刻的时候，我们会考虑使用本地缓存。但是使用本地缓存的数据一致性问题会非常严重，而我们可以尝试将一致性哈希负载均衡算法和本地缓存结合在一起，以提高缓存命中率，并且降低本地缓存的总体内存消耗。比如说针对用户的本地缓存，我们可以使用用户 ID 来计算哈希值，那么可以确保同一个用户的本地缓存必然在同一个节点上。不过即便是采用了一致性哈希负载均衡算法，依旧不能彻底解决数据一致性的问题，只能缓解一下。

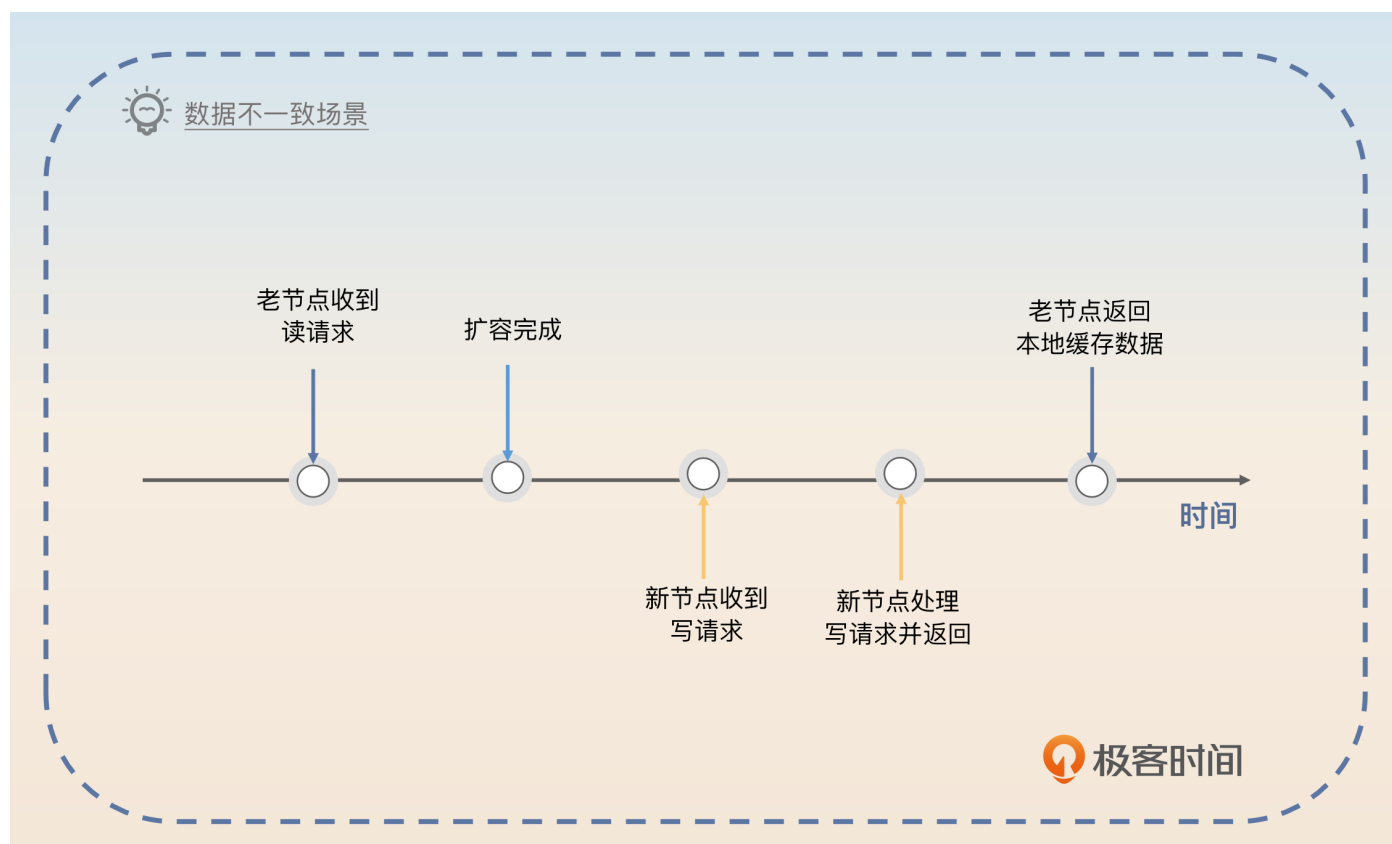
最后一句就是你留下的鱼饵。如果面试官追问为什么不能彻底解决，那么你就可以这样回答，关键词是**应用发布**。

当整个集群的节点数量发生变化的时候，就难免会导致同样的数据缓存在多个节点上。

例如在用户这个例子中，假如最开始有一个请求需要 user_id 为 1 的昵称小明，这个请求最开始会命中老节点。但是此时还没有查询到数据。紧接着扩容。此时又来了一个请求，那么它会被导去新节点。这一个请求会将 user_id

为 1 的昵称改为小刚。如果这时候第一个请求从老节点的缓存上读出了数据，那么它拿到的就还是老的数据。而应用发布是引起节点数量变化最常见的原因。毕竟应用发布可以看作先下线一个节点，再上线一个节点。

不过同时也可以看出来，在本地缓存结合了一致性哈希负载均衡算法之后数据一致性的问题已经被大大缓解了。



在这个方案中，你已经主动聊起了缓存和数据一致性的问题，那么面试官很可能就把话题转到缓存和一致性相关的问题。不过你也不用慌，在后面的内容里面我会告诉你如何应对。

面试思路总结

好了，这就是这节课的主要内容。最后我们来一起回顾一下。

这节课我们主要解决的是负载均衡的问题。我给出了负载均衡的基本算法、面试的思路和亮点方案。其中我提到几个关键词，分别是**大请求**、**成加败减**、**上下限**、**可用性**、**应用发布**。你可以从这几个关键词出发，加上自身真实的案例，梳理自己的面试思路，把这些知识内化为自己的，相信在遇到这些问题的时候，你一定可以让面试官刮目相看。

最后，我整理了这一节内容的思维导图，你可以参考一下。



思考题

如果单纯从算法效果看, 随机和轮询其实差不多。而现在据我观察, 使用轮询要比使用随机多得多, 你觉得这是为什么?

在基本算法总结里面我用最少连接数算法举了一个反面例子, 但是同样的算法用在网关负载均衡上, 就没有类似的问题, 为什么?

欢迎你把你的答案分享在评论区, 也欢迎你把这节课的内容分享给需要的朋友, 我们下节课再见!

© 版权归极客邦科技所有, 未经许可不得传播售卖。页面已增加防盗追踪, 如有侵权极客邦将依法追究其法律责任。

精选留言 14